

PhishGuard

An AI-Powered Phishing Detection and Safe Link Detonation System

Detailed Project Report

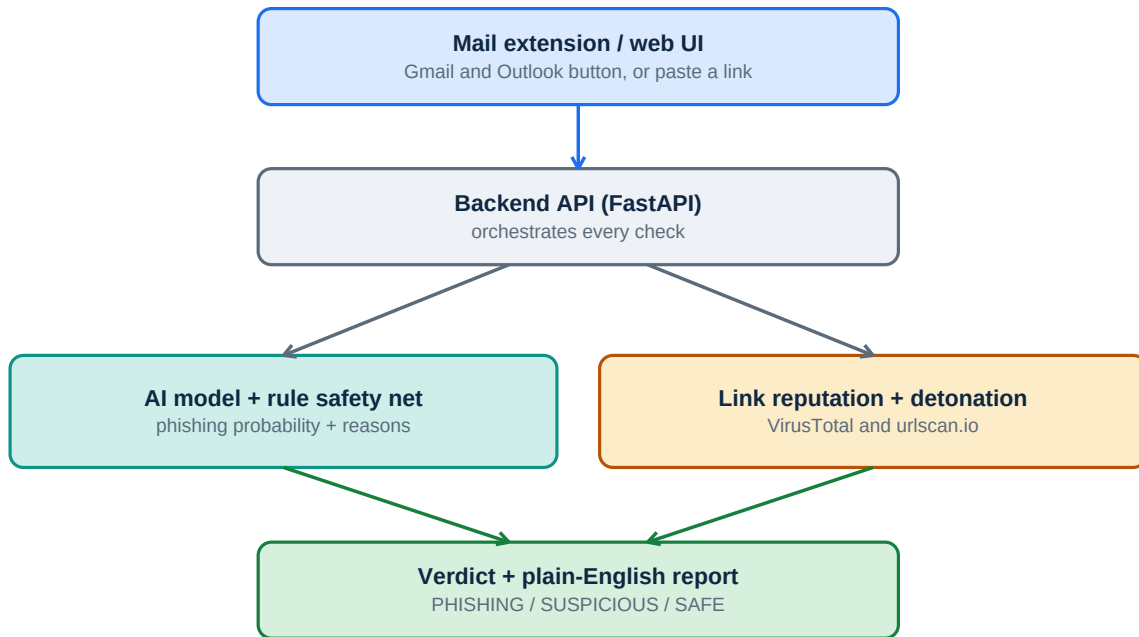


Figure 1 — Architecture of the system as actually built and used

Author	Nitin	Domain	Cybersecurity + Artificial Intelligence
Type	Individual project	Date	June 2026

Abstract

PhishGuard is an AI-powered system that detects phishing emails and safely investigates the links they contain. The working system combines a machine-learning classifier, a rule-based safety net, live link-reputation services, and sandboxed link detonation via urlscan.io, delivered as both a web application and a browser extension that runs inside Gmail and Outlook. A more ambitious self-hosted detonation sandbox on an isolated AWS EC2 instance was also attempted: the cloud infrastructure was deployed successfully, but the link-opening browser inside the isolated container did not run, so this component was not completed and the system instead uses urlscan.io. This report documents, in detail, what was built and why, what was achieved, and what was attempted unsuccessfully — including the reasons.

1. Introduction and Objectives

Phishing is one of the most common and damaging cyber-attacks: a convincing email persuades a person to click a link and hand over credentials or download malware. End users are the last line of defence at the moment they decide whether to trust a message. The objective was to build a practical, explainable tool that helps the user make that decision inside their inbox, and that can safely reveal what a suspicious link would do.

Objectives:

- Classify an email as phishing or legitimate from its sender, subject, body and links, with a clear explanation of *why*.
- Strengthen the decision with live threat-intelligence about the links.
- Open suspicious links in an isolated sandbox and report their behaviour, without risk to the user.
- Deliver it as a usable product: a web app and a Gmail/Outlook extension.

2. What Makes the Project Distinctive

Two design choices set PhishGuard apart. First, **safe link detonation**: rather than only scoring text, the system opens a link in an isolated sandbox and shows the evidence (final destination, screenshot, verdict). Second, **security at the human's point of decision**: the checks run inside the Gmail or Outlook window with a plain-language verdict, instead of a silent yes/no buried in a mail filter.

3. Methodology and Development Approach

The system was built in four phases, each adding one capability and always leaving a working product. A fifth, more advanced component (a self-hosted AWS sandbox) was attempted but not completed.

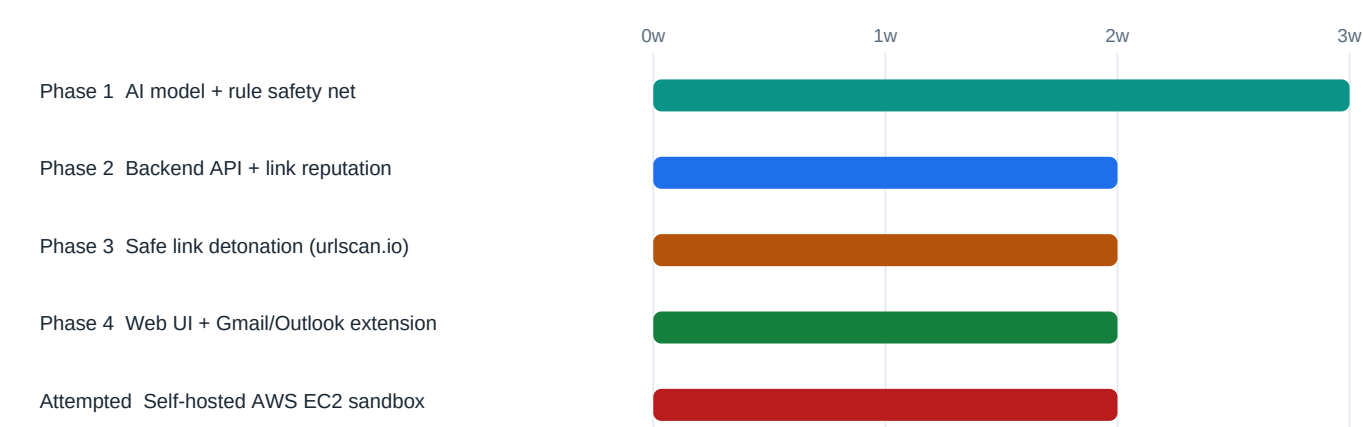


Figure 2 — Phased development timeline (the AWS sandbox was attempted, shown in red)

3.1 Technologies used and why

Component	Technology	Why it was chosen
AI model	scikit-learn Logistic Regression	Simple, fast, and explainable (each feature has a weight)
Backend API	Python + FastAPI	Modern, auto-generates interactive docs, same language as the model
Link reputation	VirusTotal, urlscan.io	Free tiers; industry-trusted threat intelligence
Live detonation	urlscan.io sandbox	Hosted, safe, no infrastructure to run (the working engine)
Web UI	HTML / CSS / JavaScript	Lightweight single page, served by the API
Extension	Chrome extension (Manifest V3)	Runs inside Gmail and Outlook; calls the local backend
Attempted cloud sandbox	AWS EC2 + VPC + Terraform	Isolated, disposable infrastructure-as-code (not completed)

4. Phase 1 — The AI Model (Used / Achieved)

The core converts an email into 19 measurable clues (features) and uses a trained Logistic Regression model to output a phishing probability with reasons. The model cannot read English, so it is given numeric signals a human would notice.



Figure 3 — The AI model pipeline, from raw email to explained verdict

4.1 Features

The 19 features fall into three groups. **URL clues:** number of links, raw-IP links, URL shorteners, the '@' redirect trick, many subdomains, suspicious top-level domains, link length, and whether a body link impersonates a known brand. **Language clues:** urgency words, threat words, credential requests, money/prize bait, a generic greeting, exclamation marks, and body length. **Sender clues:** free-email provider, display-name vs. domain mismatch, digits/hyphens in the domain, and look-alike sender domains.

4.2 Training and results

The model was trained with a 25% held-out test set and 5-fold cross-validation. For a security tool, **recall** (catching real phishing) is prioritised alongside precision.

Metric	Result (held-out test set)
Accuracy	1.000
Precision	1.000
Recall	1.000
F1 score	1.000
5-fold cross-validation F1	1.000 (plus/minus 0.000)
Confusion matrix	30/30 correct, 0 errors

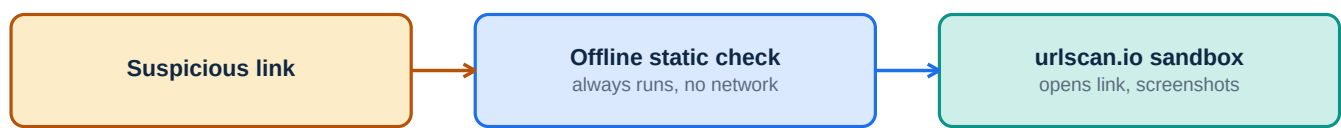
Honest note. The near-perfect scores reflect a clean, synthetic starter dataset that was clearly separable. On real public datasets (PhishTank, the Nazario corpus, Enron) the numbers would be lower and more realistic. The synthetic data proves the pipeline is correct; using real data is the next step.

5. Phase 2 — Backend API and Link Reputation (Used / Achieved)

The trained model was wrapped in a FastAPI service that accepts an email (or a pasted link, domain or raw message) and returns a combined verdict. The backend forms two independent opinions: the AI model judges the text, while VirusTotal and urlscan.io judge the links. A link confirmed malicious by threat-intelligence overrides the model and forces a PHISHING verdict. The service degrades gracefully: without API keys it still runs using the model only.

6. Phase 3 — Safe Link Detonation (Used / Achieved)

Detonation means opening a suspicious link in an isolated place to see what it really does, without it touching the user's computer. The working system uses two safe layers: an **offline static analysis** that always runs (no network, no key) and flags suspicious structure, and a **live detonation** via the urlscan.io sandbox, which opens the link and returns the final destination, a screenshot and a verdict.



The link is never opened on the user's computer; the sandbox opens it and reports back.

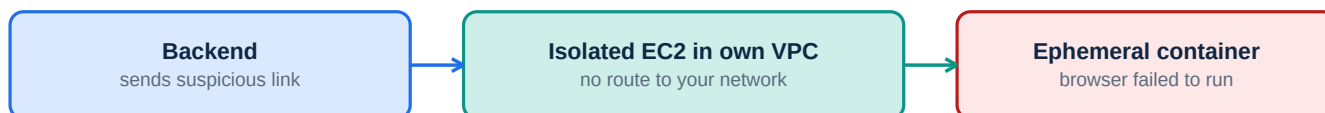
Figure 4 — Safe detonation flow used in the final system

Live detonation was verified working by detonating a real URL through urlscan.io, which returned a sandbox result. Two issues found and fixed along the way: free urlscan accounts reject 'private' scans (switched to 'unlisted'), and urlscan returns an error for domains that do not resolve (now shown as a friendly message).

7. Attempted Component — Self-Hosted AWS EC2 / VPC Sandbox (Not Completed)

To make detonation fully self-owned — so that even real malware could not reach the user's resources, and so each organisation could run the sandbox inside its own isolated cloud — an isolated detonation sandbox was designed for an AWS EC2 instance, provisioned with Terraform. The intended isolation design is shown below.

ATTEMPTED - infrastructure deployed, but in-container detonation did not run



VPC + network ACL + security group deployed via Terraform; backend connected; container browser did not launch.

Figure 5 — Intended (attempted) isolated AWS sandbox; the container browser did not run

7.1 What was achieved

- **Infrastructure-as-code.** Terraform successfully provisioned a dedicated VPC, an internet gateway and subnet, a network ACL that denies all private IP ranges, a security group admitting only the backend's IP, an IAM role, and the EC2 instance.
- **Connectivity.** The backend successfully reached the instance's dispatcher service over the network, confirming the isolation and routing design worked.

7.2 What was not achieved, and why

The actual link detonation runs inside a hardened Docker container on the instance. That container's headless browser never launched successfully. The specific failures were:

- **Missing browser library.** The container returned *ModuleNotFoundError: No module named 'playwright'* — the Playwright browser-automation package/Chromium was not correctly available inside the built image.
- **Fix attempt did not resolve it in time.** The image was rebuilt to explicitly install Playwright and Chromium and to relax the read-only container flag, and the instance was recreated; the in-container browser still did not run within the project timeframe.
- **Operational friction.** The Playwright browser image is large (~2 GB) and slow to build on a small instance; remote debugging via AWS Session Manager showed the instance still provisioning/offline, making iteration slow, and the running instance was incurring cost.

To avoid further cloud cost and complexity, the EC2 instance was decommissioned with 'terraform destroy', and urlscan.io was adopted as the working live-detonation engine. The AWS work therefore demonstrates the isolation architecture and infrastructure-as-code skills, but self-hosted detonation was **not** achieved end-to-end and is recorded here as attempted, with the reasons above.

8. Phase 4 — Web Interface and Browser Extension (Used / Achieved)

A clean web application lets the user paste a link, a domain or a whole email and receive a colour-coded verdict with reasons and per-link results. A single Chrome extension (Manifest V3) brings this into the inbox: it injects a Check email button into Gmail and Outlook on the web, reads the open message, and shows the verdict in a panel without leaving the inbox. Cross-origin access was enabled so the extension can reach the local backend.

9. Engineering Challenges Found and Fixed

Testing on realistic inputs exposed three weaknesses, each diagnosed and fixed — the most valuable learning of the project.

9.1 Short phishing emails slipped through

A one-line phishing email scored as safe because the model had only seen wordy phishing. Fixed by diversifying the training data and adding a rule safety net; the same email then scored 99.9%.

9.2 Brand impersonation hidden in body links

A clear phishing email scored only 26% because the impersonating domain was in a body link, while the model only checked the sender's domain. Fixed by adding a feature and rule to detect brand impersonation in body links; the same email then scored 100%, with no false positives on real domains.

9.3 Sandbox scan rejections

Live detonation initially failed on free urlscan accounts (private scans rejected) and on non-resolving domains. Fixed by switching to unlisted scans and a friendly message; detonation of a real URL was then verified working.

10. Summary of Outcomes (Achieved vs. Attempted)

Component	Status
AI phishing classifier with explanations	Achieved
Rule-based safety net	Achieved
Backend API combining model + link reputation	Achieved
Offline static link analysis	Achieved
Live link detonation (urlscan.io)	Achieved
Web application	Achieved
Gmail / Outlook browser extension	Achieved
AWS VPC + EC2 isolated infrastructure (Terraform)	Deployed, then decommissioned
Self-hosted in-container detonation on EC2	Attempted - not successful

11. Limitations and Future Work

- **Synthetic training data.** Retraining on large real-world datasets is the priority for realistic accuracy.
- **Self-hosted sandbox.** Getting the in-container browser working on the isolated EC2 is the main unfinished item; using a known-good Playwright base image or AWS Fargate, with more build time, would likely resolve it.
- **English-focused features.** Multilingual and obfuscated phishing need additional work.
- **Page-structure fragility.** The extension's reading of Gmail/Outlook is best-effort; an official add-on would be more robust.
- **Deployment.** Hosting the backend in the cloud and publishing the extension are future steps.

12. Skills Demonstrated

Practical machine learning (feature engineering, training, evaluation with precision and recall); API development with FastAPI; integration of third-party security APIs; secure-by-design thinking (isolation, least privilege, never trusting input); browser-extension development; and infrastructure-as-code with Terraform on AWS — including the realistic experience of an advanced component that had to be scoped out when it could not be completed in time.

13. Conclusion

PhishGuard delivers a complete, explainable defence against phishing that meets the user at the point of decision and can safely reveal what a malicious link would do, using a transparent ML model, a rule safety net, live threat-intelligence, and urlscan.io detonation, through both a web app and an in-inbox extension. An isolated self-hosted AWS sandbox was attempted and its infrastructure deployed, but the in-container detonation could not be completed and was honestly scoped out. Beyond the working system, the project's iterative discovery and repair of real weaknesses — and the candid handling of a component that did not succeed — reflect sound engineering judgement and a security mindset.